

Python for Computer Vision

Most Popular Frameworks for Deep Learning

Framework	Features	Supports languages
Tensorflow	Highly flexible system architecture	Python, C++ and R
Caffe	Speed, transposability and applicability in modelling Convolution Neural Networks (CNN)	C, C++, Python, MATLAB
CNTK	Easy training and combination of popular model types across servers	Python, C++ and the Command Line Interface
Torch/PyTorch	The entire deep modeling process is far more simpler as well as transparent	Lua, Python
Keras	Provide a simplistic interface for the purpose of quick prototyping by constructing effective neural networks that can work with TensorFlow	Python

Computer vision tasks:

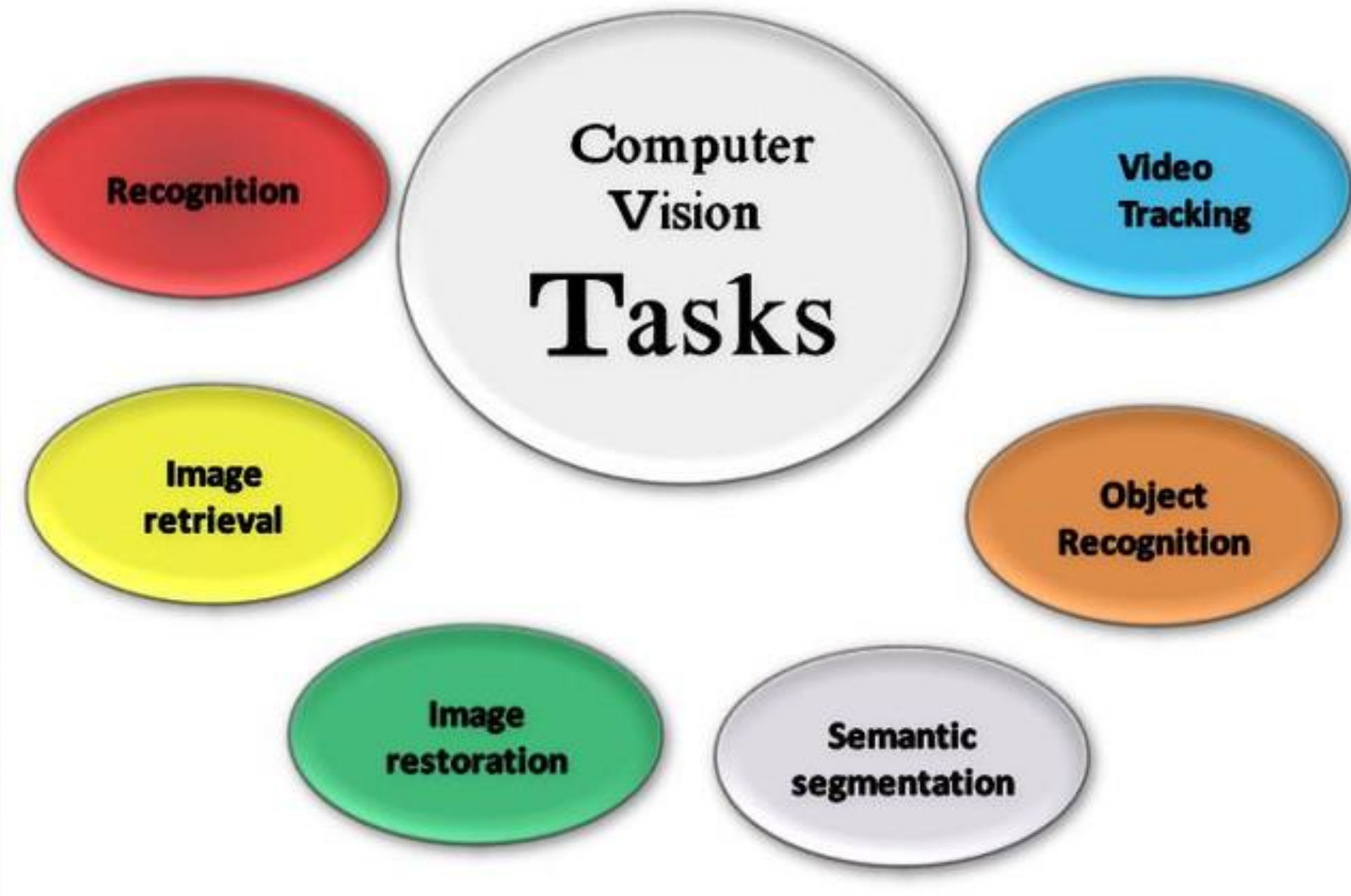


Image and Numpy Basics

- Numpy can't open directly jpeg file so we need to use PIL (Python Imaging Library)

- `import numpy as np`
- `import matplotlib.pyplot as plt`
- `%matplotlib inline`
- `from PIL import Image`
- `pic = Image.open('00-puppy.jpg')`
- `pic`

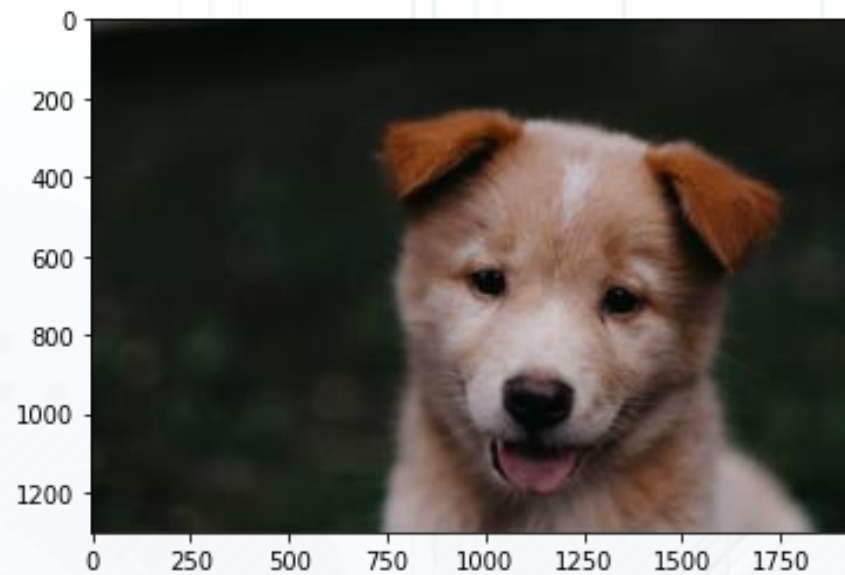


Image and Numpy Basics

- `type(pic)`
- `PIL.JpegImagePlugin.JpegImageFile`
- So this Jpeg file so NumPy can't function on such extension files so we need to create the image into array
- `pic_arr = np.asarray(pic)`
- `type(pic_arr) ->` `numpy.ndarray`
- `pic_arr.shape ->` `(1300, 1950, 3)`
- So this image has 1300 pixel width, 1950 pixel height and 3 color channels

Image and Numpy Basics

```
plt.imshow(pic_arr)
```



```
# Accessing one of the channel
```

```
pic_red = pic_arr.copy()
```

```
pic_green = pic_arr.copy()
```

```
pic_blue = pic_arr.copy()
```

```
#zero out green and blue channel # R G B - 0 1 2
```

```
plt.imshow(pic_red[:, :, 0])
```

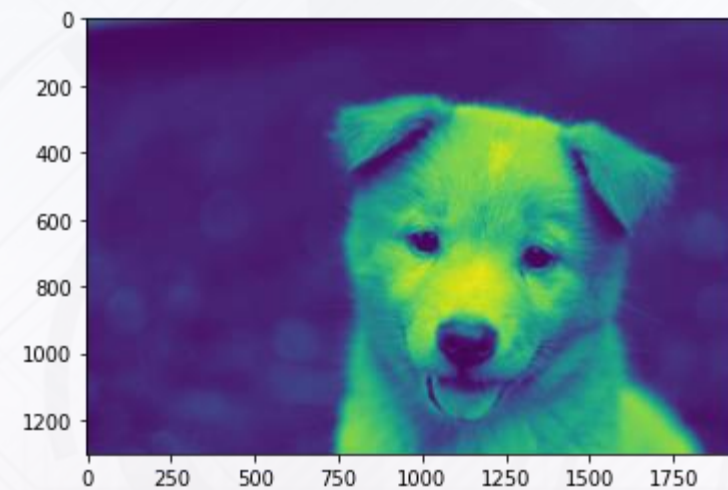
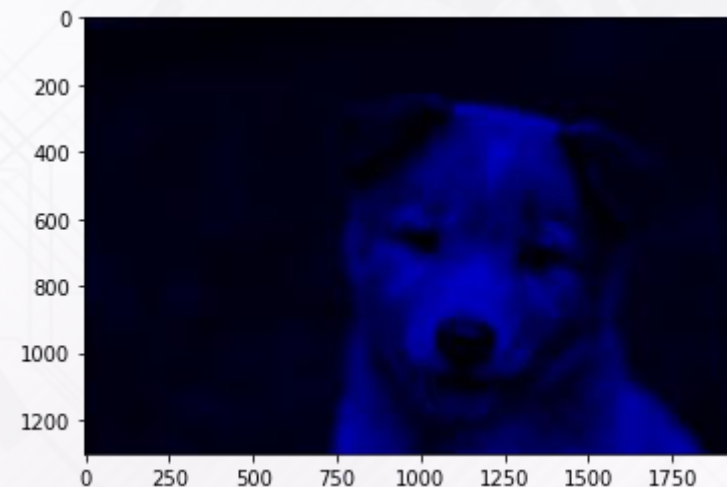
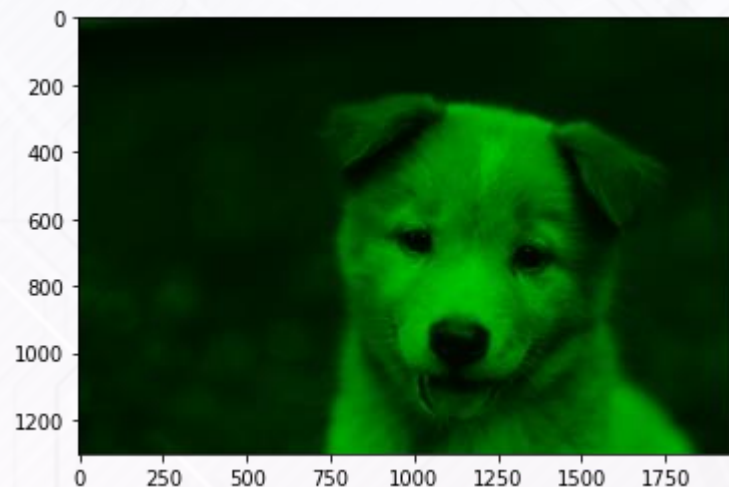
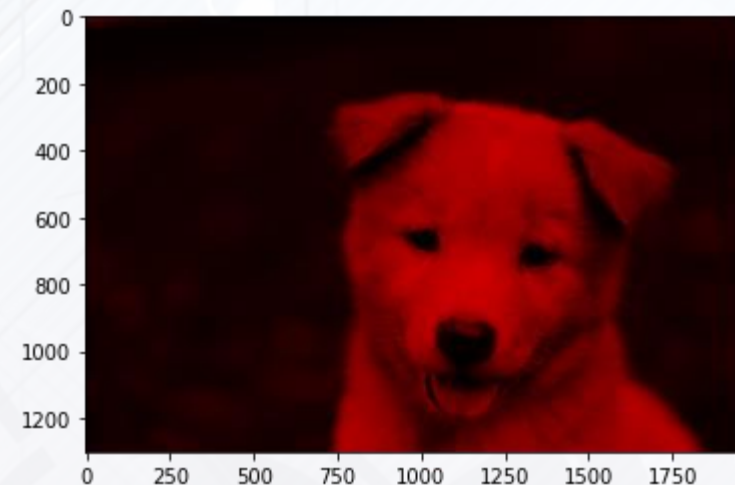


Image and Numpy Basics

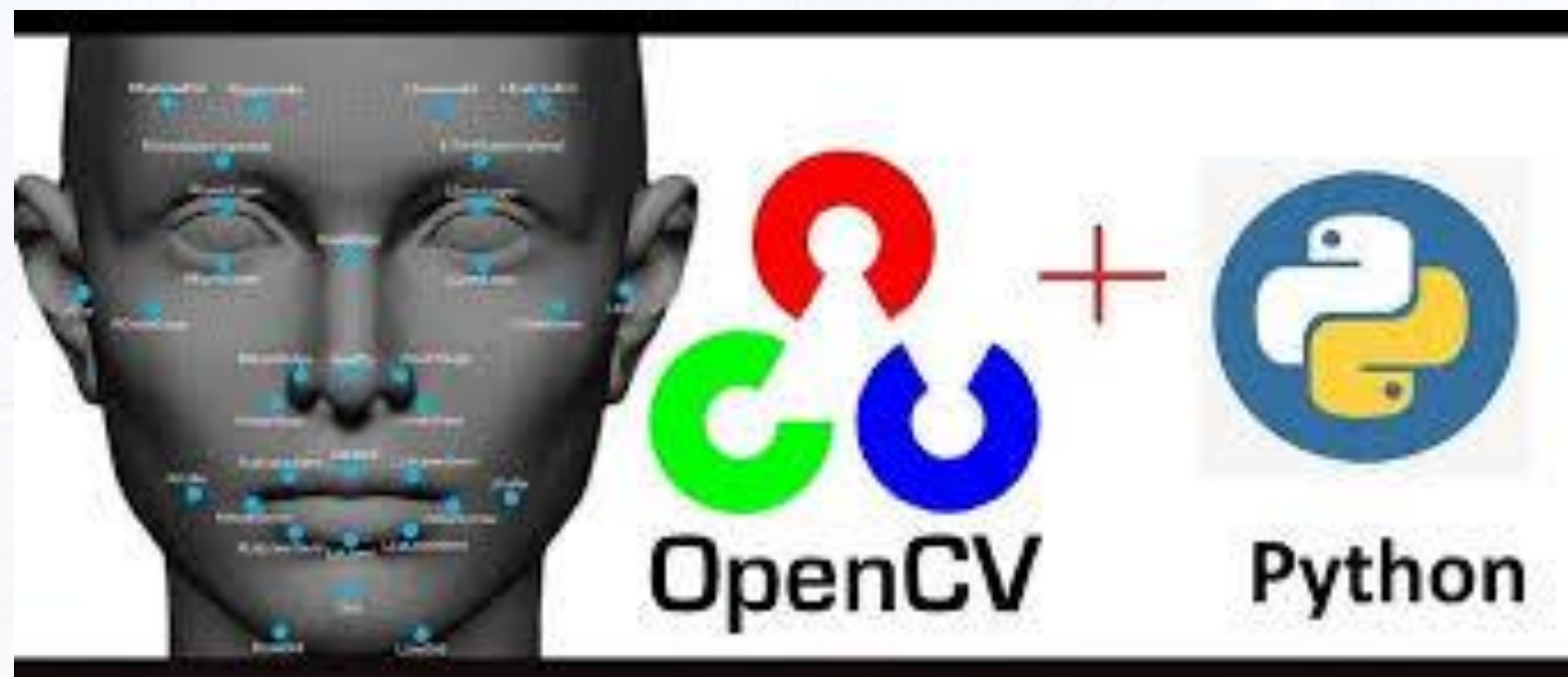
However the image above for red doesn't show red image has function between [0-255] but matplotlib function will show image by default like this with color scale

```
#how to see red channel only without matplotlib default coloring function  
pic_red[:, :, 1]=0 #green channel is zero  
  
pic_red[:, :, 2]=0 # blue channel is zero  
  
plt.imshow(pic_red)
```



OpenCV:

OpenCV (Open Source Computer Vision Library) is a library of programming functions mainly for real-time computer vision. Originally developed by Intel.



Open image files with OpenCV

```
import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline

# importing openCV libraries

import cv2

img = cv2.imread('00-puppy.jpg')

type(img) #directly reads it as numpy array and no need to convert it to
array          numpy.ndarray

plt.imshow(img)
```



Now while reading an image using openCV, it shows image in weird bluish tint. This is because OpenCV and matplotlib expects **different orders of RGB**

Open image files with OpenCV

MATPLOTLIB --> RGB - Red Green Blue

OPENCV --> BGR - Blue Green Red

So we need to transform from BGR to RGB

```
fix_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(fix_img)
```

```
# resize by ratio
```

```
w_ratio = 0.5 h_ratio = 0.5 #reduce image by 50%
```

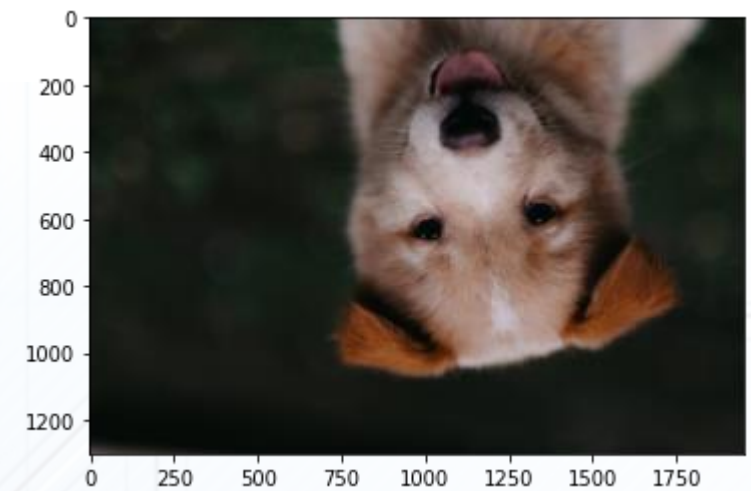
```
new_img = cv2.resize(fix_img, (0,0), fix_img, w_ratio, h_ratio)  #(0,0 location from where you want to resize)
```

```
plt.imshow(new_img)
```



Open image files with OpenCV

```
flip_img = cv2.flip(fix_img, 0)  
  
plt.imshow(flip_img) #horizontal
```

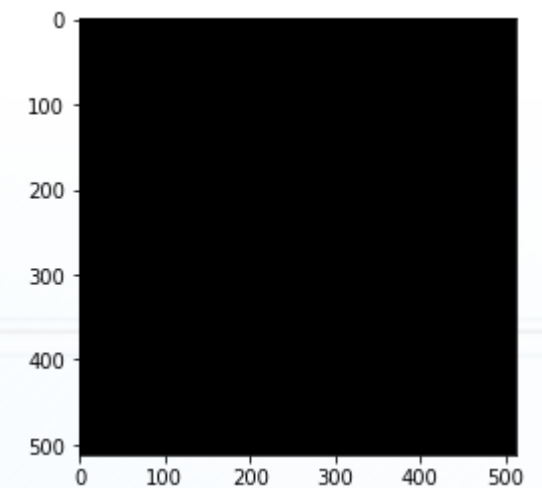


Drawing on Images

```
blank_img = np.zeros(shape=(512, 512, 3), dtype=np.int16)
```

```
blank_img.shape      (512, 512, 3)
```

```
plt.imshow(blank_img)
```

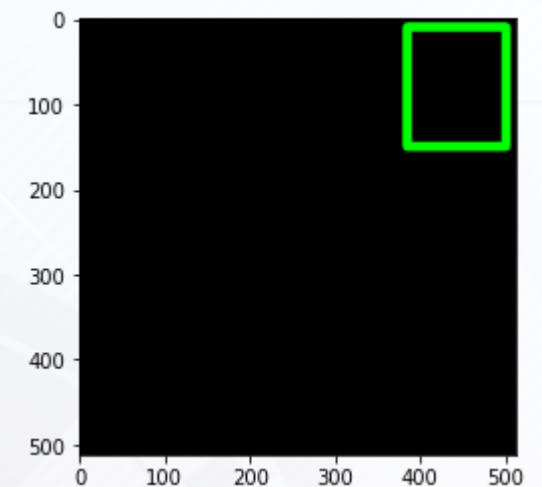


```
## Adding rectangle on blank image (permanent marker to blank image)
```

```
cv2.rectangle(blank_img, pt1=(384, 10), pt2=(500, 150), color=(0, 255, 0), thickness=10)
```

```
# pt1 - top left corner # pt2 - bottom right corner
```

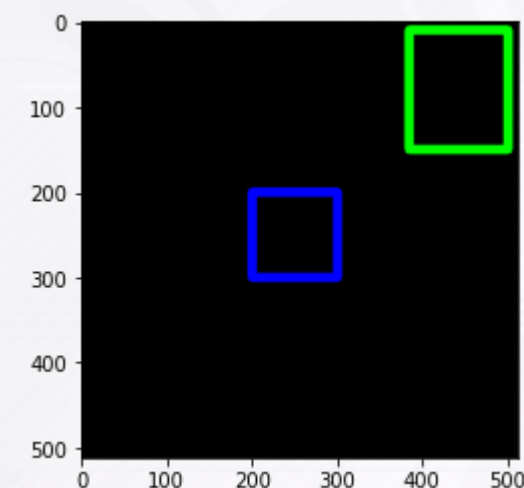
```
plt.imshow(blank_img)
```



```
## Drawing a square in the middle
```

```
cv2.rectangle(blank_img, pt1=(200, 200), pt2=(300, 300), color=(0, 0, 255), thickness=10)
```

```
plt.imshow(blank_img)
```



Open image files with OpenCV

```
## Draw a circle
```

```
cv2.circle(img=blank_img,center=(100,100),radius=50,  
color=(255,0,0),thickness=10) plt.imshow(blank_img)
```

```
## Filled in circle or rectangle (thickness =-1)
```

```
cv2.circle(img=blank_img,center=(400,400),radius=50,  
color=(255,0,0),thickness=-1) plt.imshow(blank_img)
```

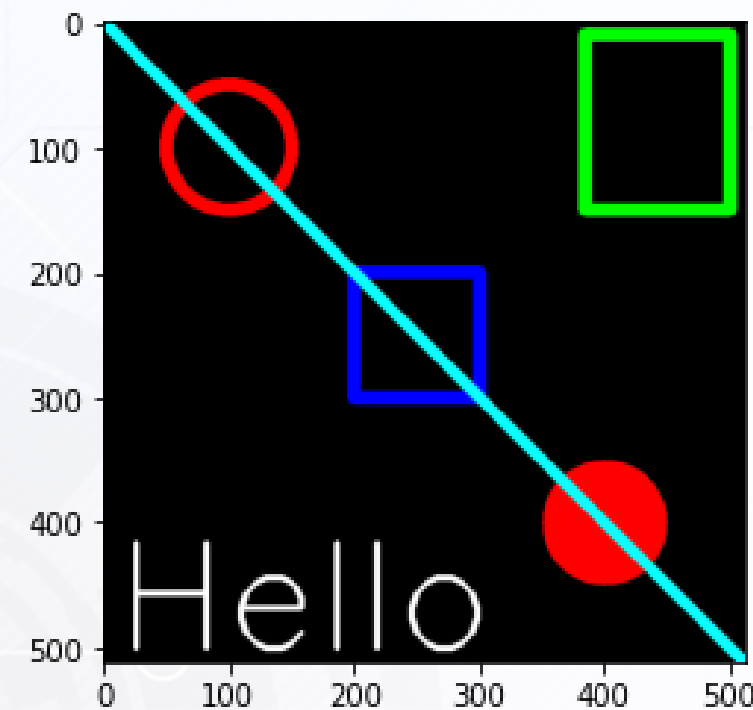
```
## Draw a line
```

```
cv2.line(blank_img,pt1=(0,0),pt2=(512,512),  
color=(10,255,255),thickness=8) plt.imshow(blank_img)
```

```
# Now we will write and draw polygons on blank image
```

```
font = cv2.FONT_HERSHEY_SIMPLEX #choose font  
cv2.putText(blank_img,text='Hello',org=(10,500),  
fontFace=font,fontScale=4,color=(255,255,255), thickness=3,  
lineType=cv2.LINE_AA) #org is bottom left starting point so need to start  
x,y from there
```

```
plt.imshow(blank_img)
```



Open image files with OpenCV

```
import cv2

img = cv2.imread("flower.jpg")

def draw_circle(event, x, y, flags, param):

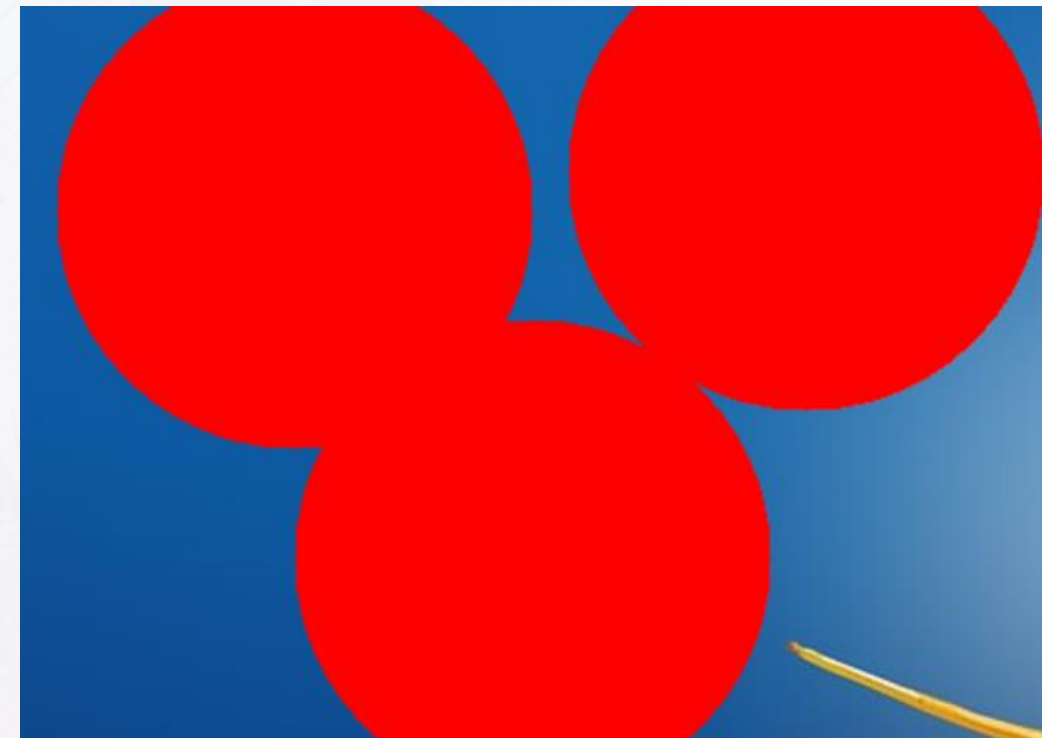
    if event == cv2.EVENT_LBUTTONDOWN:
        print("hello")
        cv2.circle(img, (x, y), 100, (0, 255, 0), -1)

cv2.namedWindow(winname = "Title of Popup Window")
cv2.setMouseCallback("Title of Popup Window", draw_circle)

while True:
    cv2.imshow("Title of Popup Window", img)

    if cv2.waitKey(10) & 0xFF == 27:
        break

cv2.destroyAllWindows()
```



Basics of OpenCV

1) Reading an Image

```
1 img = cv2.imread("C:\\Users\\Darshita\\Desktop\\OpenCV\\flower.jpg")
```

cv2.imread command reads an Image from the given path and stores it in a form of a matrix.

Note - When the image is read with the OpenCV function imread(), the order of colors is **BGR (blue, green, red)**.

Shape of an Image

```
1 imgRGB.shape
```

```
(200, 200, 3)
```

In the above output (200,200,3) - the first two values denotes the width and height of the image and 3 denotes the number of color channels(one for each Blue, Green, Red) the image has.

```
1 # shape of a grayscale image  
2 gray.shape
```

Displaying an Image

```
1 img = cv2.imread("flower.jpg")  
2 cv2.imshow("Flower", img)  
3 cv2.waitKey(0)  
4 cv2.destroyAllWindows()
```



Displaying an Image using Matplotlib

```
1 plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x2636a9f5630>
```



Converting BGR image into RGB image

```
imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```



Input BGR image



Output RGB image

Converting RGB image into Grayscale

```
1 # convert image to grayscale
2 gray = cv2.cvtColor(imgRGB, cv2.COLOR_RGB2GRAY)|
3 # show image
4 plt.imshow(gray, cmap='gray')
```



Input RGB Image



Output Grayscale Image

Saving an Image

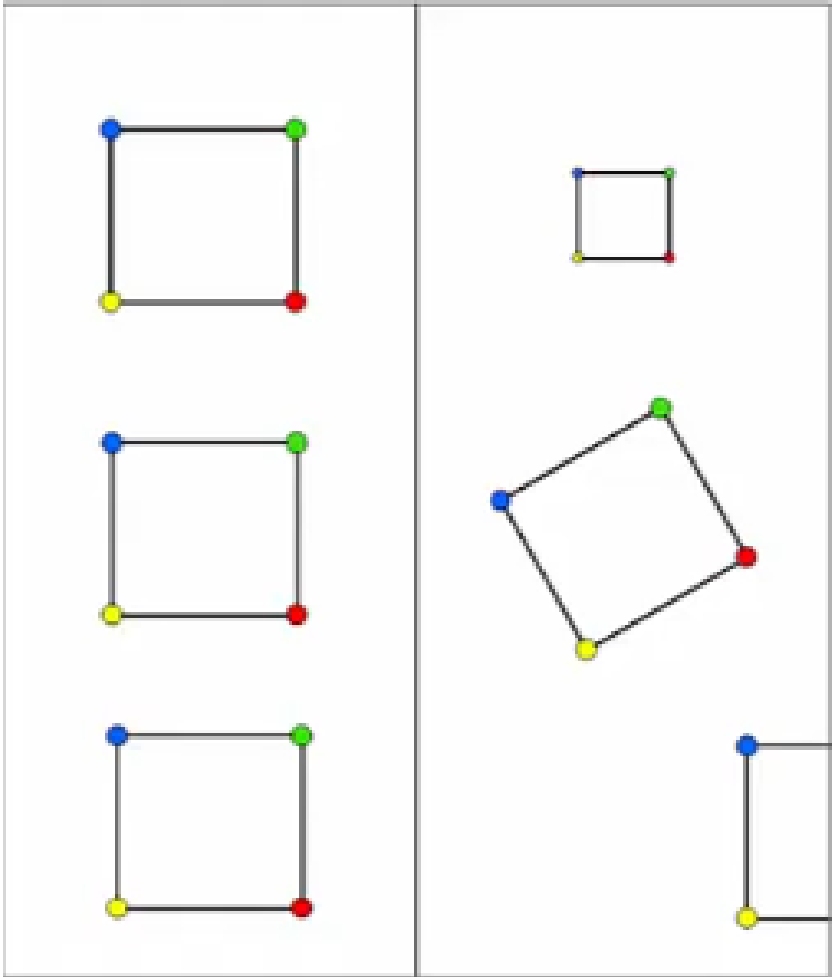
```
1 saveImg = cv2.imwrite("gray.jpg", gray)
```

```
1 # saving an image in a different format  
2  
3 imgPng = cv2.imwrite("gray.png", gray)
```

Image Transformations

1. Geometric distortions enacted upon an Image.
2. Two main types-
 - a. **Affine**- Preserves parallelism, straight lines and angles. (Scaling, Rotation, Translation etc.)
 - b. **Non- Affine** - Preserves Collinearity and Incidence. (Homography)

Affine Transformations



Non- Affine Transformations

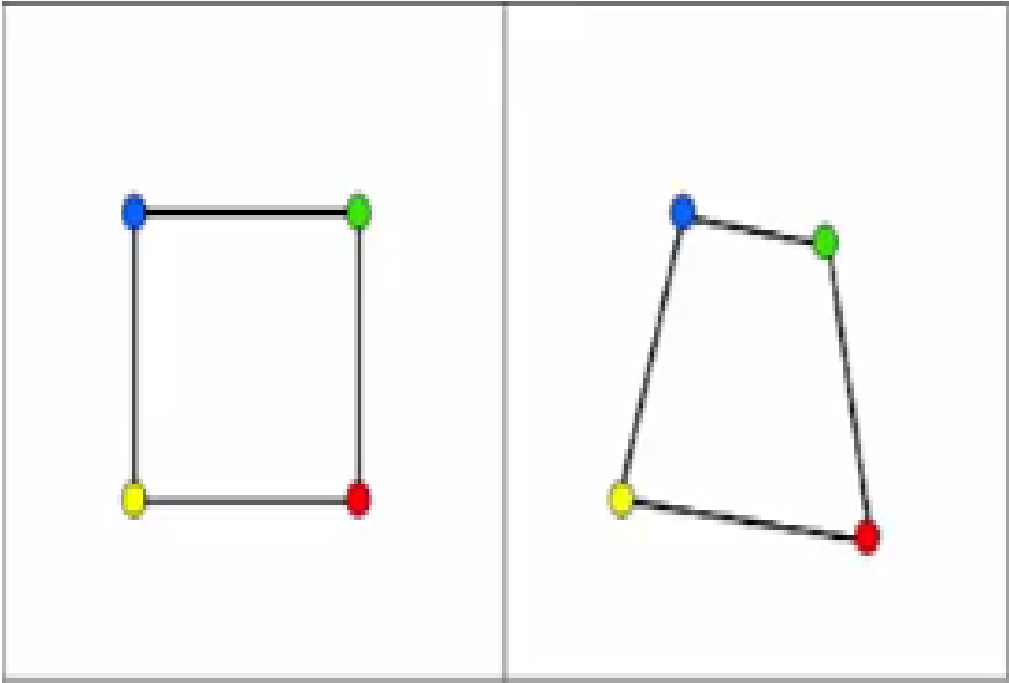


Image Translation

Moving an image in one direction (left, right, top, down).

Translation Matrix , $T = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \end{bmatrix}$

T_x - denotes the shift along x axis

T_y - denotes the shift along y axis

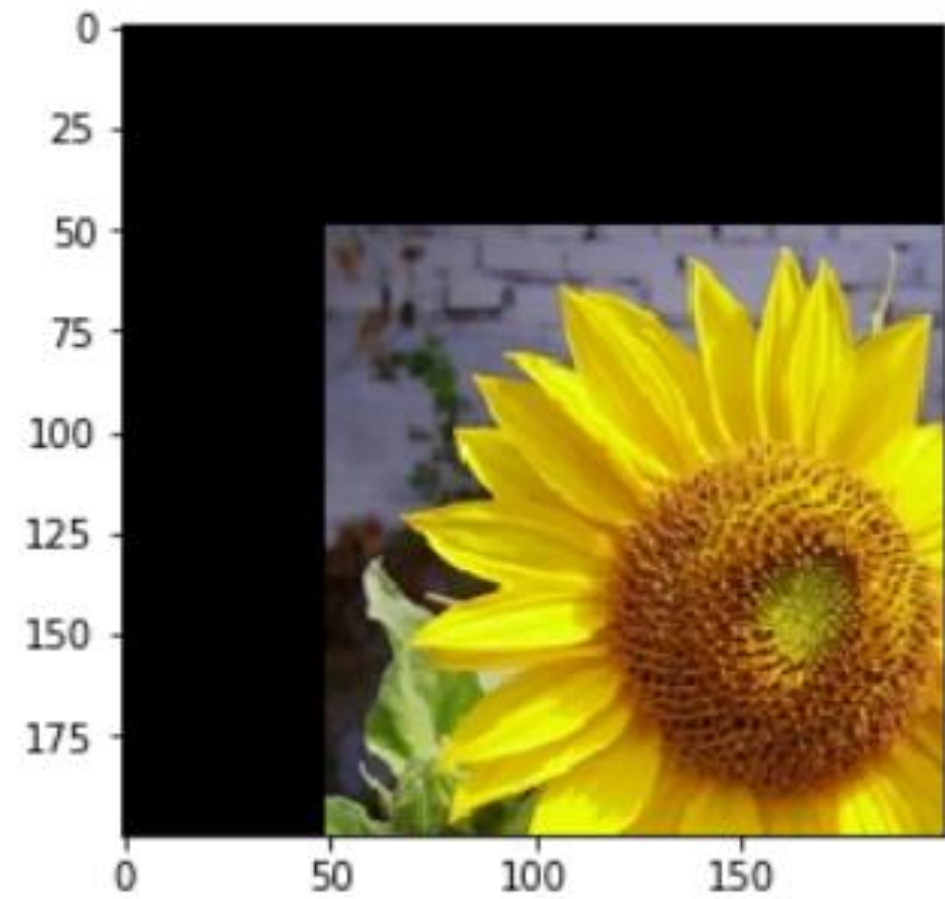
Image Translation

```
1 height , width = imgRGB.shape[:2]
2 Tx , Ty = height/4 , width/4
3
4 # Translation matrix is of the form -
5 #           | 1  0  Tx|
6 #      T =  | 0  1  Ty|
7
8 T = np.float32([[1,0,Tx],[0,1,Ty]])
9 img_trans = cv2.warpAffine(imgRGB, T , (width,height))
10 plt.imshow(img_trans)
```

Image Translation



Input Image



Translated Output Image

Image Rotation

$$\text{Rotation Matrix, } R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Where θ is the angle of rotation.

OpenCV provides us with a function for computing the above rotation matrix provided θ .

`cv2.getRotationMatrix2D((rotation_center_x, rotation_center_y), angle of rotation, scale)`

Image Rotation

To generate the rotation matrix use -

`cv2.getRotationMatrix2D((rotation_center_x, rotation_center_y) , angle of rotation, scale)`

```
1 height , width = imgRGB.shape[:2]
2 rotation_matrix = cv2.getRotationMatrix2D((width/2,height/2), 45, 1)
3 img_rotated = cv2.warpAffine(imgRGB, rotation_matrix , (width,height))
4 plt.imshow(img_rotated)
```

Image Rotation



Input Image

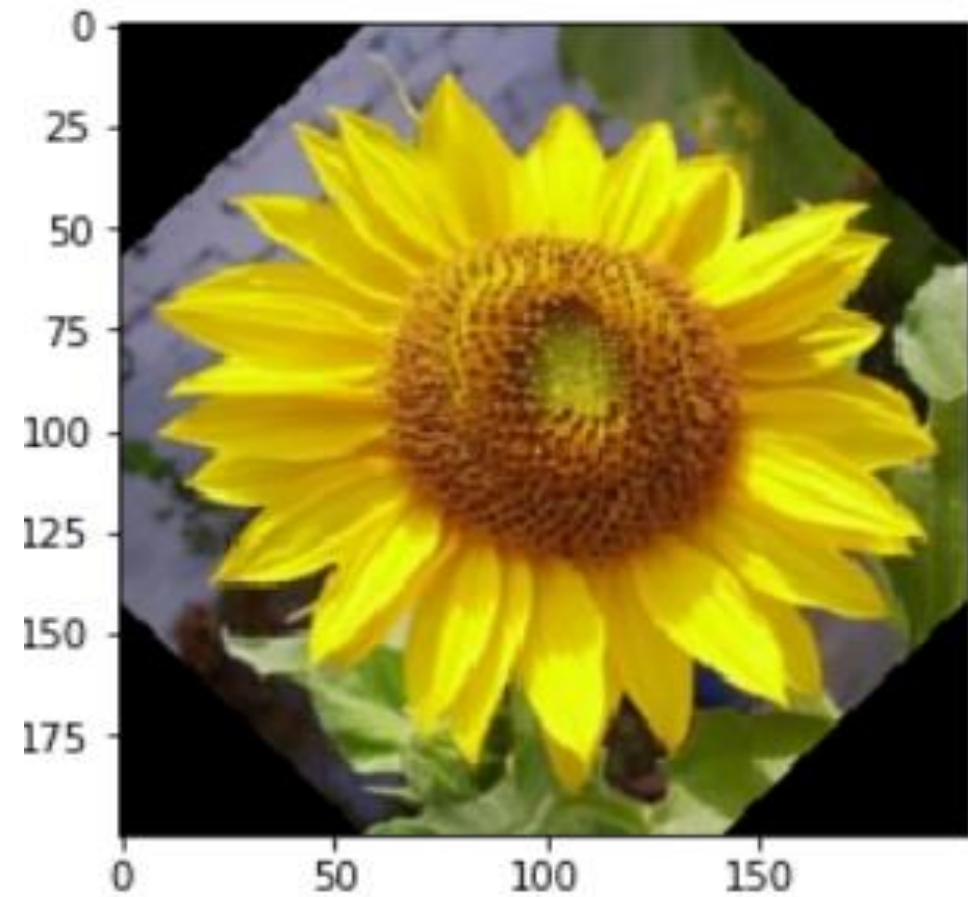


Image Rotated by 45 degrees

Image Scaling

What is Interpolation?

Interpolations means to estimate the value of unknown pixels by making use of the known image pixels. It is required when we resize an image.

`cv2.INTER_AREA` - Good for Shrinking an image

`cv2.INTER_LINEAR` - Good for Zooming an image

Image Scaling

`cv2.resize(image, output_image_size, x_scale, y_scale, interpolation)`

```
1 img_scaled = cv2.resize(imgRGB, None, fx = 1.5, fy = 1.5)
2 plt.imshow(img_scaled)
```

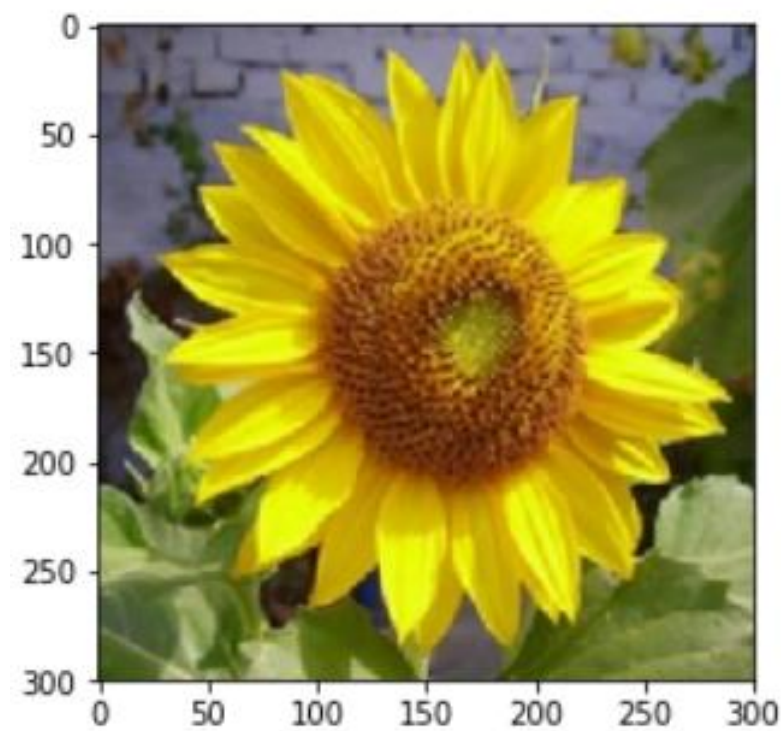


Image Smoothing

```
1 # use cv2.GaussianBlur to smooth the image
2 blurred = cv2.GaussianBlur(imgRGB, (11,11),0)
3 plt.imshow(blurred)
```



Input Image

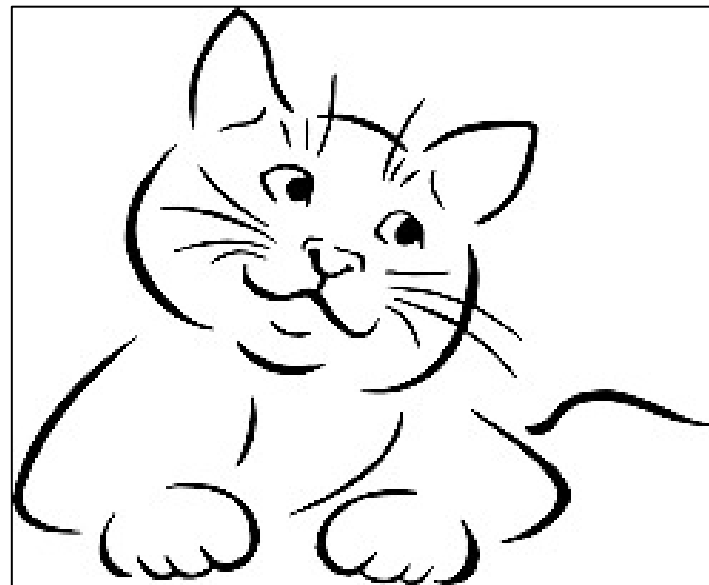


Output Image

Edges in an Image

Why edges are important?

- Edges preserves a lot of image even though a lot of detail is missing.



Edge Detection

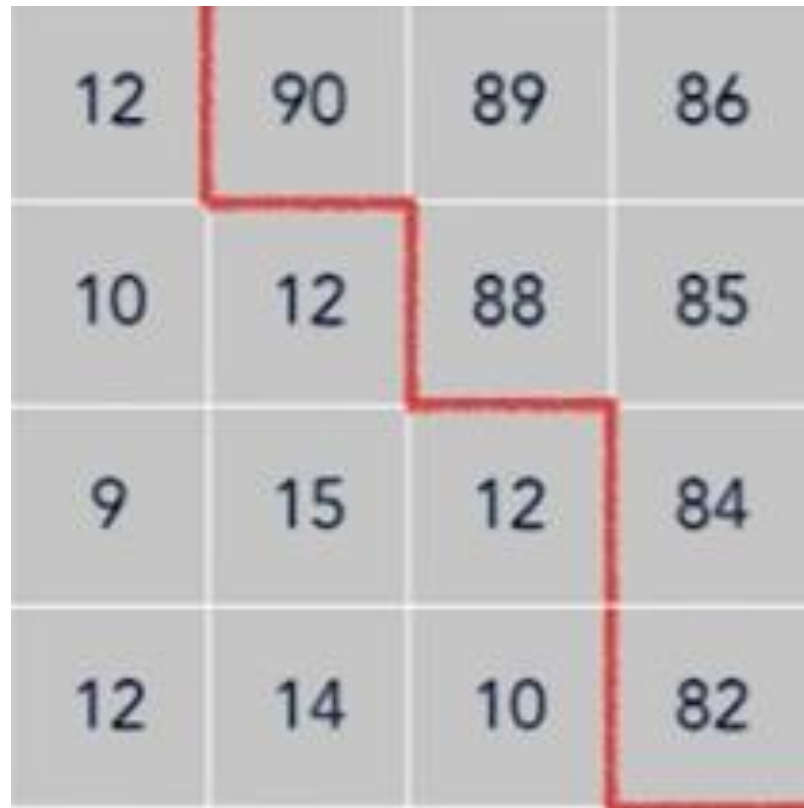
Basic Intuition is to look for neighborhood with strong magnitude of change.

12	90	89	86
10	12	88	85
9	15	12	84
12	14	10	82

12	90	89	86
10	12	88	85
9	15	12	84
12	14	10	82

Edge Detection

Basic Intuition is to look for neighborhood with strong magnitude of change.



Canny Edge Detection Algorithm

- 1) Applies Gaussian Blurring
- 2) Finds intensity gradient of an image
- 3) Apply non- maximum suppression (i.e. removes pixels that are not edges)
- 4) Hysteresis - Applies threshold (i.e. if a pixel is within the upper and lower thresholds, it is considered an edge)

Edge Detection

```
1 # Edge detection using Canny Edge Detection Algorithm
2 img_canny = cv2.Canny(img, 20,70)
3 plt.imshow(img_canny)
```

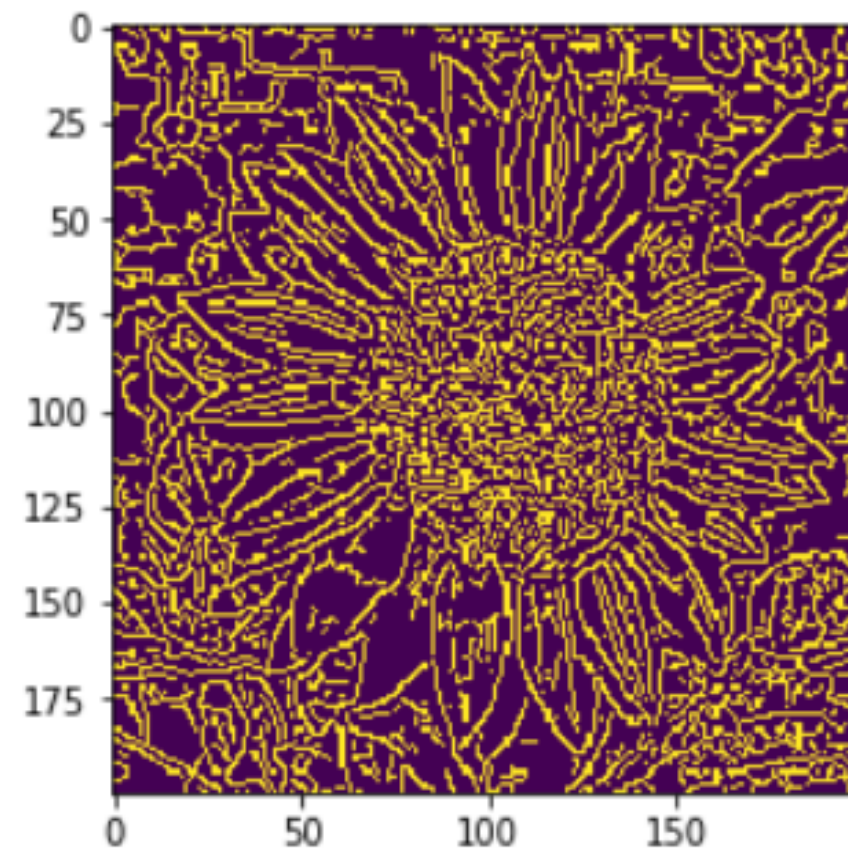
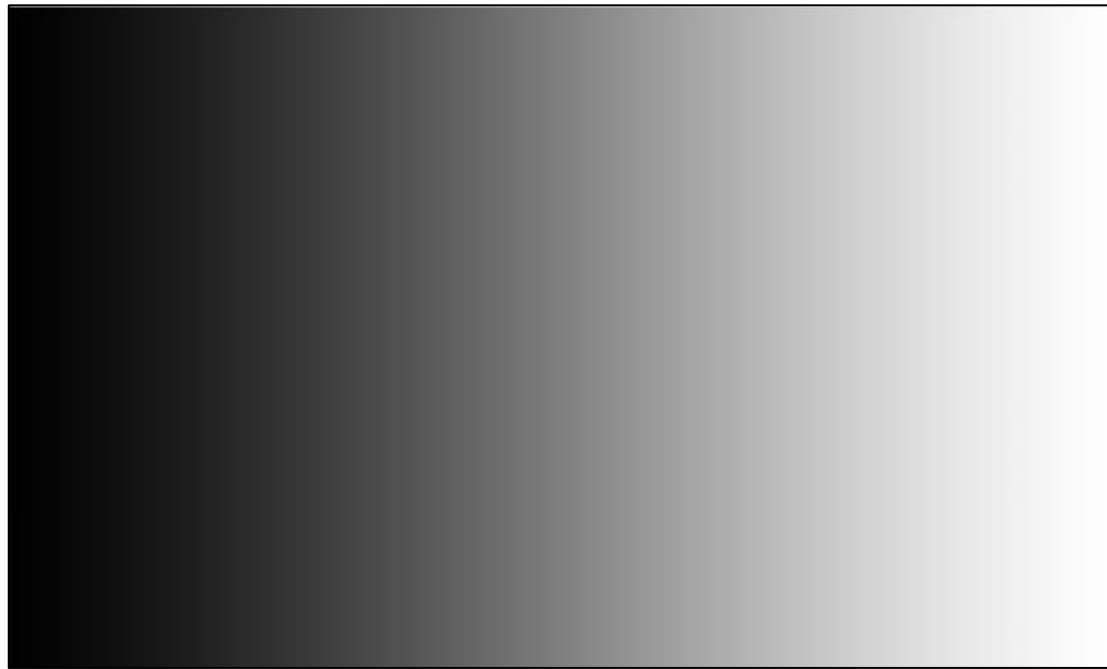
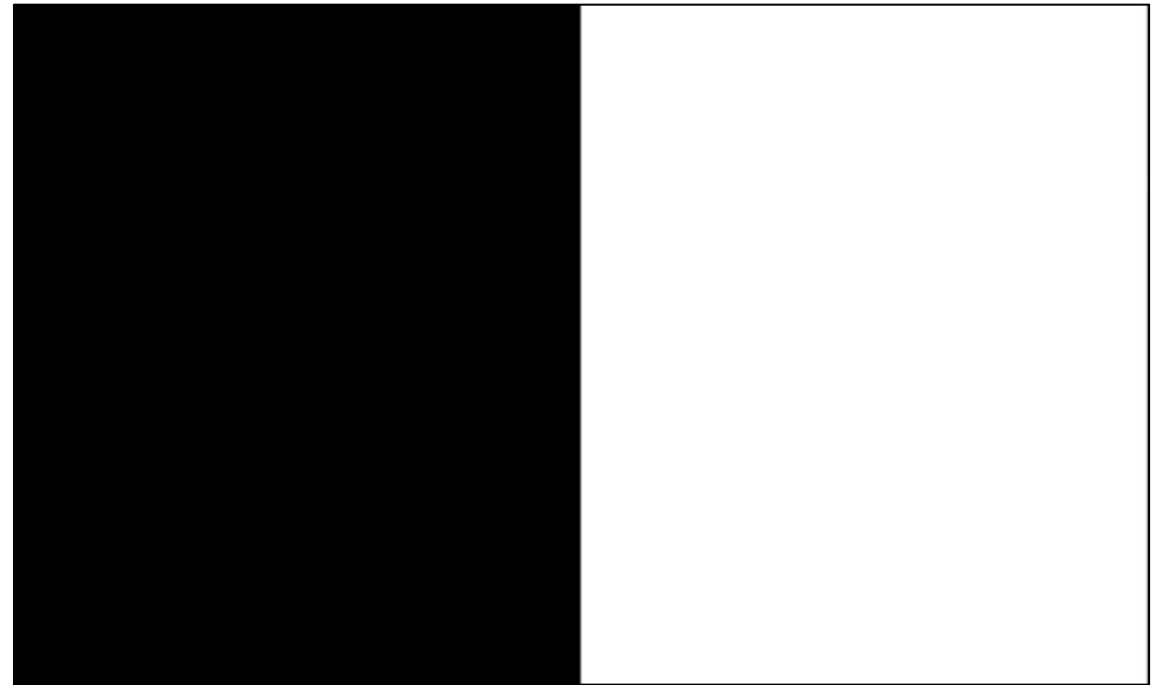


Image Thresholding



Grayscale Image



Binary Image

Image Thresholding

```
1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('flower.jpg',0)
6 ret,thresh1 = cv2.threshold(img,127,255,cv2.THRESH_BINARY)
7 ret,thresh2 = cv2.threshold(img,127,255,cv2.THRESH_BINARY_INV)
8
9 titles = ['Original Image','BINARY','BINARY_INV']
10 images = [img, thresh1, thresh2]
11
12 for i in range(3):
13     plt.subplot(1,3,i+1),plt.imshow(images[i],'gray')
14     plt.title(titles[i])
15     plt.xticks([]),plt.yticks([])
16
17 plt.show()
```

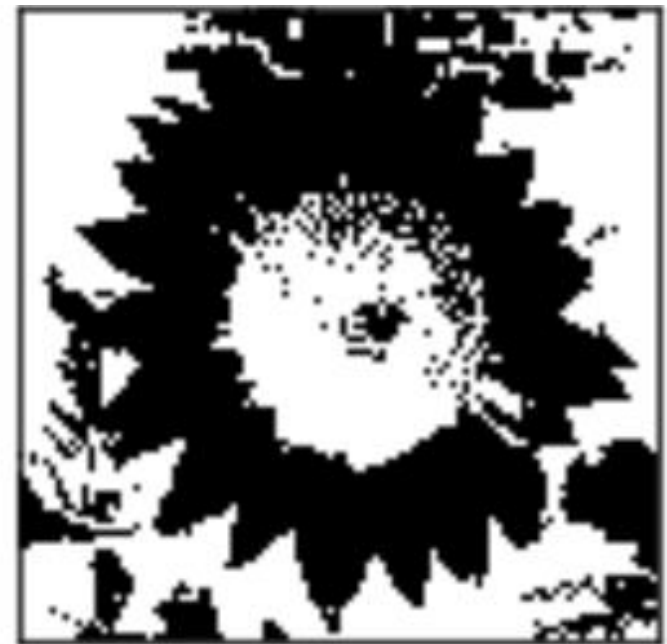
Image Thresholding



Input Image



Binary



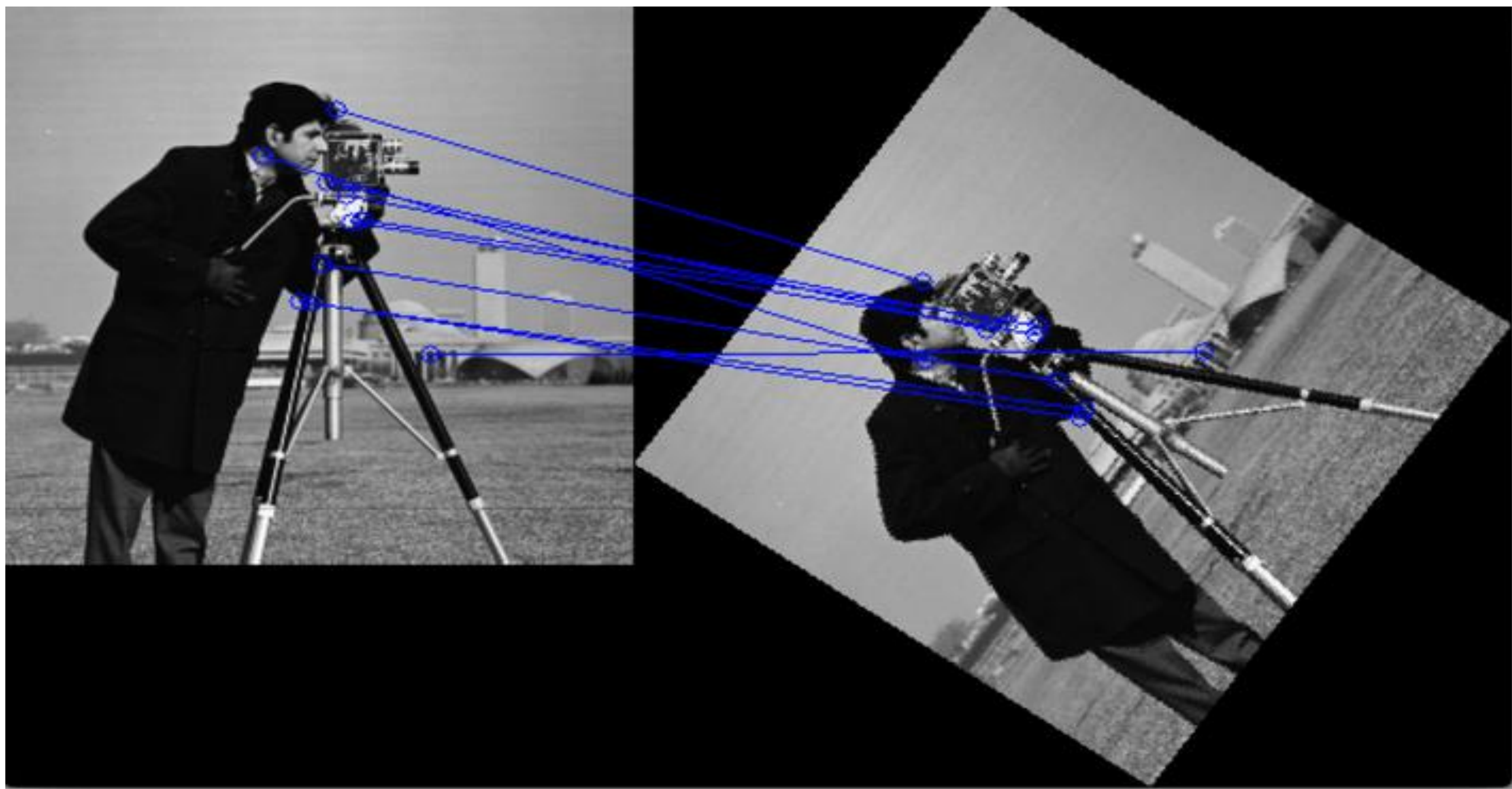
Binary-INV

Extracting Key Points from an Image

Key points - The points in an image that are invariant to Image Transformations (rotation, scaling, translation etc.).

Even after transforming an image we will be able to find the same key points in the modified as well as original image.

Use cases - Image Matching, Panorama stitching etc.



ORB (Oriented Fast Rotated Brief)

```
1 import numpy as np
2 import cv2
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('C:\\Users\\Darshita\\Desktop\\OpenCV\\flower.jpg')
6 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
7 # Initiate STAR detector
8 orb = cv2.ORB_create()
9
10 #find the keypoints and compute the descriptors with ORB
11 kp, des = orb.detectAndCompute(img, None)
12
13 # draw keypoints location
14 img2 = cv2.drawKeypoints(img, kp, None)
15 plt.imshow(img2), plt.show()
```

ORB Feature Extractor



Panorama Stitching

Step 1 - Detect the key points in both the input images.

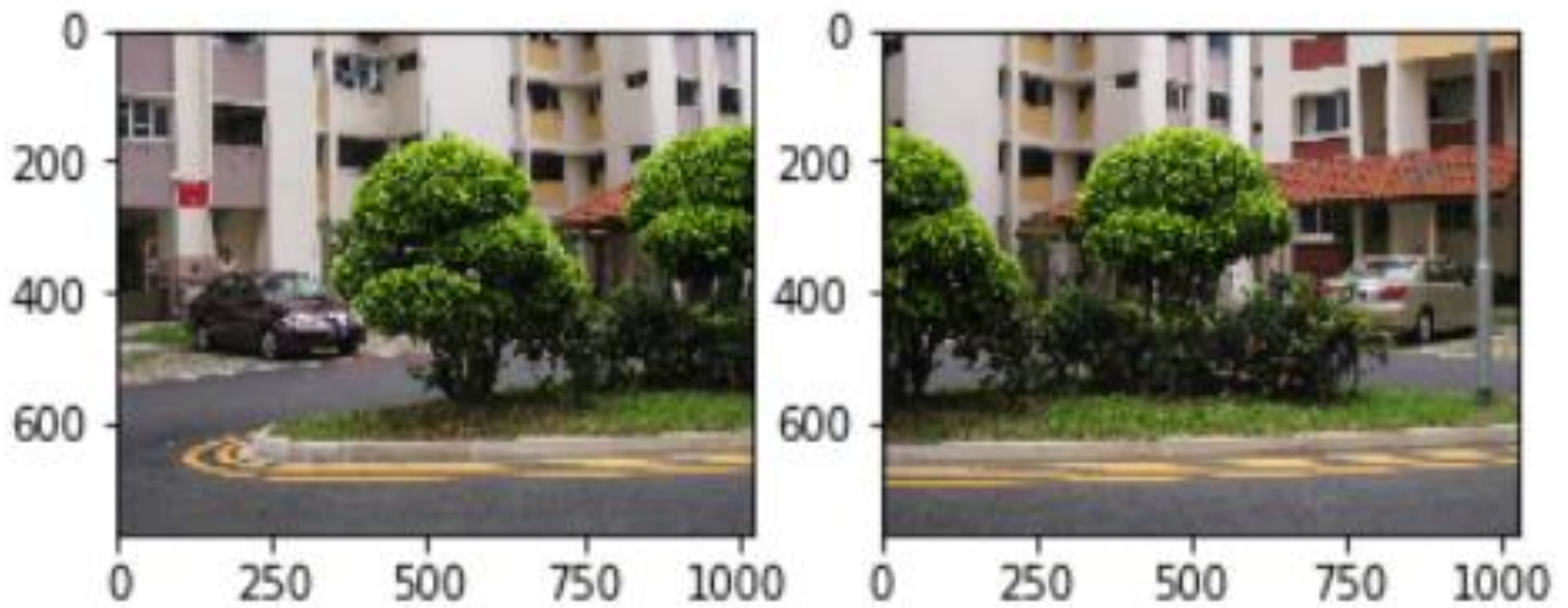
Step 2 - Match Keypoints between the two images to be stitched.

Step 3 - Stitch the two images if they have matching key points otherwise the images cannot be stitched.

Panorama Stitching

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # read the two input image, convert it into RGB scale and resize for further operations
6 dim=(1024,768)
7 left=cv2.imread('img1.jpg',cv2.IMREAD_COLOR)
8 left=cv2.resize(left,dim,interpolation = cv2.INTER_AREA) #ReSize to (1024,768)
9 left1 = cv2.cvtColor(left,cv2.COLOR_BGR2RGB)
10
11 right=cv2.imread('img2.jpg',cv2.IMREAD_COLOR)
12 right=cv2.resize(right,dim,interpolation = cv2.INTER_AREA) #ReSize to (1024,768)
13 right1 = cv2.cvtColor(right,cv2.COLOR_BGR2RGB)
14
15 # for plotting the two input images side by side
16 f, axarr = plt.subplots(1,2)
17 axarr[0].imshow(left1)
18 axarr[1].imshow(right1)
19
```

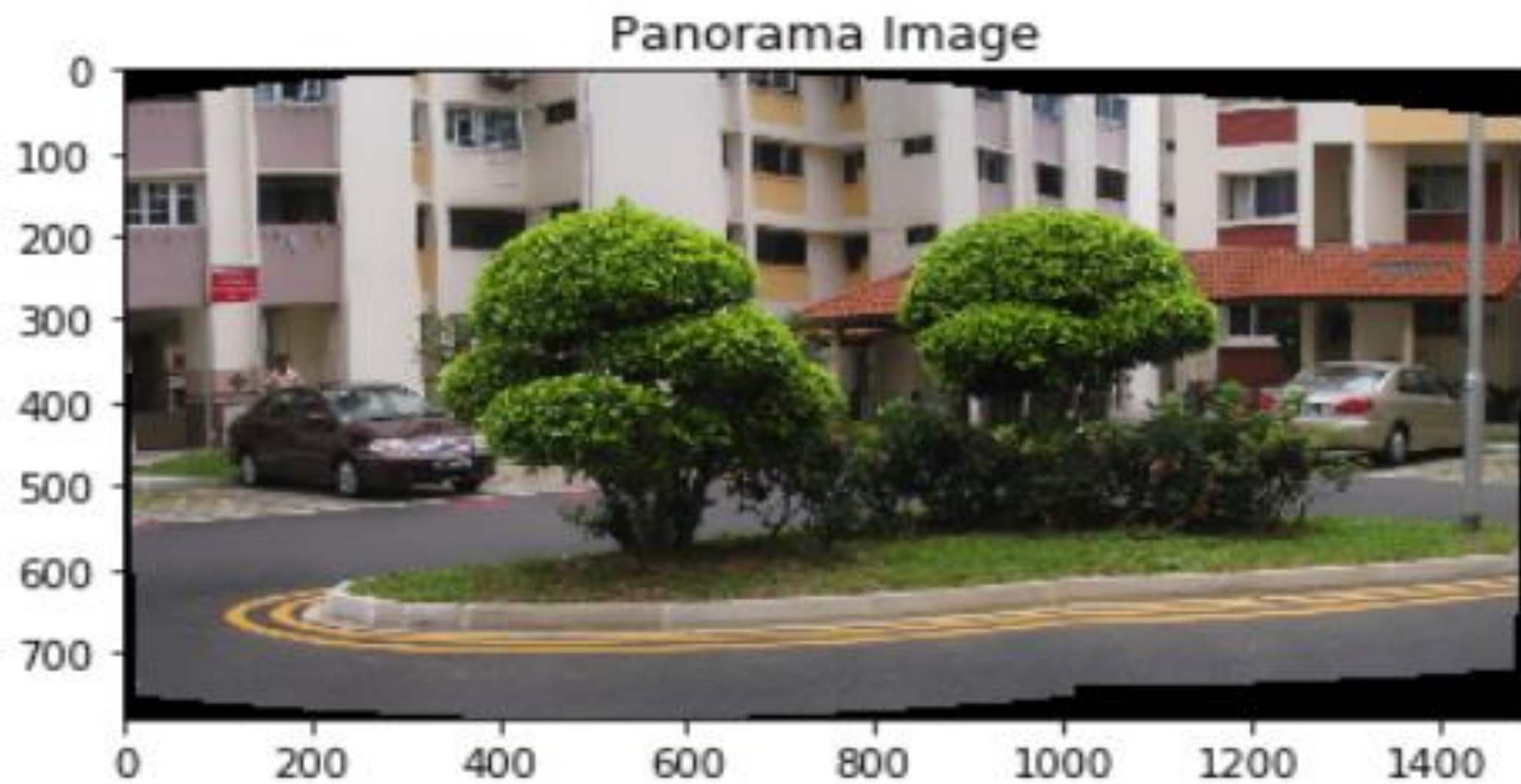
Panorama Stitching



Panorama Stitching

```
1  # create a list which will contain the two input images
2  images=[]
3  images.append(left)
4  images.append(right)
5
6  #stitcher = cv2.createStitcher()
7  stitcher = cv2.Stitcher.create()
8
9  # ret contains True if pano contains a valid output else False
10 ret,pano = stitcher.stitch(images)
11
12 if ret==cv2.STITCHER_OK:
13     pano = cv2.cvtColor(pano,cv2.COLOR_BGR2RGB)
14     plt.title('Panorama Image')
15     plt.imshow(pano)
16
17 else:
18     print("Given images cannot be stitched!")
```

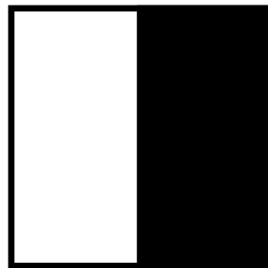
Panorama Stitching



Face and Eye Detector

Haar Cascade Classifier –

A machine learning based object detection method that makes use of Haar features to identify objects in an image.



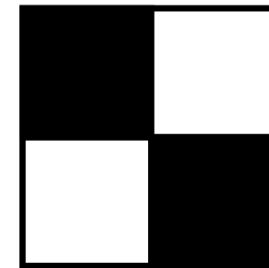
(1)



(2)



(3)



(4)

Face and Eye Detector

- Haar cascade classifiers are trained by using lots of positive and negative examples.



Negative Example



Positive Example

- For detailed explanation of Haar cascade classifiers please



luzpaz Some mist. typo fixes ...

Latest commit 413fba1 on Feb 7, 2018

..

 haarcascade_eye.xml	some attempts to tune the performance	6 years ago
 haarcascade_eye_tree_eyeglasses.xml	some attempts to tune the performance	6 years ago
 haarcascade_frontalcatface.xml	fixing models to resolve XML violation issue	3 years ago
 haarcascade_frontalcatface_extende...	fixing models to resolve XML violation issue	3 years ago
 haarcascade_frontalface_alt.xml	some attempts to tune the performance	6 years ago
 haarcascade_frontalface_alt2.xml	some attempts to tune the performance	6 years ago
 haarcascade_frontalface_alt_tree.xml	some attempts to tune the performance	6 years ago
 haarcascade_frontalface_default.xml	some attempts to tune the performance	6 years ago
 haarcascade_fullbody.xml	Some mist. typo fixes	2 years ago
 haarcascade_lefteye_2splits.xml	some attempts to tune the performance	6 years ago
 haarcascade_licence_plate_rus_16sta...	Added Haar cascade for russian cars licence plate detection, 16 stage...	6 years ago
 haarcascade_lowerbody.xml	Some mist. typo fixes	2 years ago
 haarcascade_profileface.xml	some attempts to tune the performance	6 years ago
 haarcascade_righteye_2splits.xml	some attempts to tune the performance	6 years ago
 haarcascade_russian_plate_number....	Create haarcascade_russian_plate_number.xml	6 years ago
 haarcascade_smile.xml	fixing models to resolve XML violation issue	3 years ago
 haarcascade_upperbody.xml	Some mist. typo fixes	2 years ago

[Link to the repo!](#)

Face and Eye Detector

```
1  #import required libraries
2  import numpy as np
3  import cv2 as cv
4  import matplotlib.pyplot as plt
5  %matplotlib inline
6  |
7  #load the classifiers downloaded from the Github repo
8  face_cascade = cv.CascadeClassifier('haarCascadeFrontalFaceDefault.xml')
9  eye_cascade = cv.CascadeClassifier('haarCascadeEyes.xml')
10
11 #read the image and convert to grayscale format
12 img = cv.imread('face.jpg')
13 inputImg = cv.cvtColor(img, cv.COLOR_BGR2RGB)
14 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
15 gray = cv.cvtColor(img, cv.COLOR_RGB2GRAY)
16
```

Face and Eye Detector

```
17 #calculate coordinates of the bounding boxes for face
18 faces = face_cascade.detectMultiScale(gray)
19 for (x,y,w,h) in faces:
20     cv.rectangle(img, (x,y), (x+w,y+h), (255,0,0), 2)
21     # to ensure that the classifier detects eyes within the face and not outside it.
22     roi_gray = gray[y:y+h, x:x+w]
23     roi_color = img[y:y+h, x:x+w]
24     eyes = eye_cascade.detectMultiScale(roi_gray)
25
26     #draw bounding boxes around detected features
27     for (ex,ey,ew,eh) in eyes:
28         cv.rectangle(roi_color, (ex,ey), (ex+ew,ey+eh), (0,255,0), 2)
29
30 # plot the input and output image
31 f, axarr = plt.subplots(1,2)
32 axarr[0].imshow(inputImg)
33 axarr[1].imshow(img)
34
35 #write image
36 cv2.imwrite('face_detection.jpg',img)
```

Face and Eye Detector

